

Chương 3: Đối tượng dữ liệu tuần tự

Nhóm A: Iterable và sequence

Bài 1. Slicing dữ liệu chuỗi thời gian

Cho danh sách nhiệt độ 24 giờ:

```
temperature = [  
    26.1, 25.8, 25.5, 25.2, 25.0, 25.4,  
    26.0, 27.1, 28.3, 29.4, 30.2, 31.0,  
    31.5, 32.0, 32.4, 32.1, 31.7, 30.8,  
    29.9, 29.2, 28.7, 28.0, 27.2, 26.6  
]
```

Yêu cầu:

- Tách dữ liệu buổi sáng: từ 06:00 đến 11:00.
- Tách dữ liệu buổi chiều: từ 12:00 đến 17:00.
- Lấy mỗi 3 giờ một giá trị.
- Đảo ngược dãy nhiệt độ.
- Tìm chênh lệch giữa 6 giờ đầu và 6 giờ cuối.
- Không dùng vòng lặp cho các thao tác slicing.

Bài 2. Chuẩn hóa sequence đầu vào

Viết hàm: `to_list(data)`

Yêu cầu:

- Nếu data là list, trả về bản sao list.
- Nếu data là tuple, set, range hoặc generator, chuyển về list.
- Nếu data là string, chuyển thành list các từ thay vì list ký tự:

"Python is powerful"

Kết quả: ["Python", "is", "powerful"]

- Nếu data không iterable, phát sinh `TypeError`.

Nhóm B: Hàm trên đối tượng tuần tự

Bài 3. Phân tích danh sách số

Viết hàm: `sequence_summary(values)`

Trả về:

```
{  
    "count": ...,  
    "sum": ...,  
    "mean": ...,  
    "min": ...,  
    "max": ...,  
    "sorted_asc": ...,  
    "sorted_desc": ...  
}
```

Yêu cầu:

- Không thay đổi list đầu vào.
- Phát sinh `ValueError` nếu `values` rỗng.
- Kiểm tra mọi phần tử phải là số.
- Viết docstring đầy đủ.

Bài 4. Dữ liệu nhiều trạm

Cho dữ liệu:

```
station_ids = ["A01", "A02", "A03", "A04", "A05"]
```

```
station_names = [  
    "Tan Son Hoa",  
    "Thu Duc",  
    "Bien Hoa",  
    "Nha Be",  
    "Cu Chi"
```

]

```
pm25_values = [32.5, 18.2, 41.8, 25.0, 28.6]
```

Yêu cầu:

1. Dùng `zip()` tạo các tuple (id, name, pm25).
2. Dùng `enumerate(..., start=1)` để in bản ghi có số thứ tự.
3. Dùng `dict(zip(...))` để tạo dictionary `station_id → pm25`.
4. Tìm trạm có PM2.5 cao nhất.
5. Tạo list tên trạm có PM2.5 lớn hơn hoặc bằng 25.

Bài 5. Kiểm tra tính hợp lệ bằng `any()` và `all()`

Cho: `raw_pm25 = [12.5, 18.2, -999, None, 25.4, 40.1, "NA"]`

Yêu cầu:

- Kiểm tra có giá trị âm không.
- Kiểm tra có ít nhất một giá trị không phải số không.
- Kiểm tra toàn bộ phần tử có phải số hợp lệ và không âm hay không.
- Viết generator expression cho từng điều kiện.
- Giải thích sự khác nhau giữa `any()` và `all()` trong bài toán này.

Gợi ý: `isinstance(value, (int, float))`

Bài 6. `zip_longest()` khi dữ liệu thiếu

Cho:

```
stations = ["A01", "A02", "A03", "A04"]
```

```
pm25 = [18.2, 32.5]
```

```
temperature = [29.1, 30.0, 31.2]
```

Yêu cầu:

- Dùng `itertools.zip_longest`.
- Ghép ba dãy thành bản ghi.
- Điền "MISSING" vào dữ liệu thiếu.
- Tạo list record theo dạng:

```
("A01", 18.2, 29.1)
```

- Tìm các record có ít nhất một trường bị thiếu.

Bài 7. Sắp xếp sequence phức hợp

Cho dữ liệu:

```
records = [
    ("A01", "Tan Son Hoa", 32.5, "2026-09-15 08:00"),
    ("A02", "Thu Duc", 18.2, "2026-09-15 08:00"),
    ("A03", "Bien Hoa", 41.8, "2026-09-15 08:00"),
    ("A04", "Nha Be", 32.5, "2026-09-15 09:00")
]
```

Yêu cầu:

- Sắp xếp theo PM2.5 giảm dần.
- Nếu PM2.5 bằng nhau, sắp xếp theo thời gian tăng dần.
- Tạo list chỉ chứa mã trạm theo thứ tự đã sắp xếp.
- Không thay đổi records gốc.
- So sánh `sorted(records, ...)` và `records.sort(...)`.

Nhóm C: Iterator

Bài 8. Khám phá vòng đời iterator

Cho:

```
values = [10, 20, 30, 40, 50]
```

Yêu cầu:

1. Tạo iterator = iter(values).
2. Gọi next(iterator) hai lần.
3. Dùng vòng lặp for để in phần tử còn lại.
4. Chuyển iterator còn lại thành list.
5. Thử duyệt iterator thêm lần nữa.
6. Tạo iterator mới và giải thích khác biệt.

Bài 9. Iterator cửa sổ trượt

Viết class:

```
SlidingWindow(values, window_size)
```

Ví dụ:

```
values = [10, 20, 30, 40, 50]
```

```
print(list(SlidingWindow(values, 3)))
```

Kết quả mong đợi:

```
[
    (10, 20, 30),
    (20, 30, 40),
    (30, 40, 50)
]
```

Yêu cầu:

- Nếu window_size <= 0, phát sinh ValueError.
- Nếu window_size > len(values), trả về iterator rỗng.
- Có thể dùng slicing bên trong __next__().

Ứng dụng mở rộng:

- Tính trung bình trượt cho chuỗi PM2.5.

Bài 10. Iterator đọc lô dữ liệu

Viết class:

`BatchIterator(values, batch_size)`

Ví dụ:

```
values = [1, 2, 3, 4, 5, 6, 7]
```

```
print(list(BatchIterator(values, 3)))
```

Kết quả:

```
[  
    [1, 2, 3],  
    [4, 5, 6],  
    [7]  
]
```

Yêu cầu:

- `batch_size` phải lớn hơn 0.
- Không làm thay đổi `values` gốc.
- Bài mở rộng: thêm tham số `drop_last=False`; nếu `True`, bỏ batch cuối không đủ số phần tử.

Nhóm D: Generator

Bài 11. Generator số nguyên tố

Viết generator:

`prime_numbers(limit)`

Yêu cầu:

- Sinh lần lượt các số nguyên tố từ 2 đến limit.
- Không tạo sẵn toàn bộ danh sách nguyên tố.
- Viết hàm hỗ trợ: `is_prime(n)`

Ví dụ:

```
print(list(prime_numbers(30)))
```

Kết quả:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Bài 12. Generator dãy thời gian

Viết generator:

`time_range(start_hour, end_hour, step=1)`

Ví dụ:

```
print(list(time_range(0, 24, step=3)))
```

Kết quả:

```
[0, 3, 6, 9, 12, 15, 18, 21, 24]
```

Yêu cầu:

- Kiểm tra `step > 0`.
- Nếu `start_hour > end_hour`, phát sinh `ValueError`.
- Bài mở rộng: trả về chuỗi định dạng `"00:00"`, `"03:00"`.

Bài 13. Generator xử lý log cảm biến

Cho dữ liệu:

```
lines = [  
    "A01,2026-09-15 08:00,32.5",
```

```
"A02,2026-09-15 08:00,18.2",  
"INVALID_LINE",  
"A03,2026-09-15 08:00,41.8",  
"A04,2026-09-15 08:00,not_available"  
]
```

Viết generator: `parse_records(lines)`

Yêu cầu:

- Sinh từng dictionary hợp lệ:
- ```
{
 "station_id": "A01",
 "timestamp": "2026-09-15 08:00",
 "pm25": 32.5
}
```
- Bỏ qua dòng sai định dạng.
  - Bỏ qua PM2.5 không thể chuyển về float.
  - Không dừng toàn bộ generator khi gặp dòng lỗi.

#### **Bài 14. Pipeline generator làm sạch dữ liệu**

Cho:

```
raw_values = [
 "12.5",
 "-999",
 "18.0",
 "",
 "NA",
 "35.2",
 "-1",
 "42.6",
 "20.1"
```

]

Viết pipeline gồm ít nhất ba generator:

`to_float(values)`

`valid_values(values)`

`high_values(values, threshold)`

Mục tiêu:

1. `to_float`: chuyển các chuỗi hợp lệ sang float.
2. `valid_values`: chỉ giữ giá trị không âm.
3. `high_values`: chỉ giữ giá trị lớn hơn ngưỡng.
4. Dùng pipeline để lấy các giá trị lớn hơn 25.
5. Tính trung bình chỉ bằng cách duyệt generator.

### **Bài 15. Generator trung bình trượt**

Viết generator: `moving_average(values, window_size)`

Ví dụ:

`values = [10, 20, 30, 40, 50]`

`print(list(moving_average(values, 3)))`

Kết quả:

`[20.0, 30.0, 40.0]`

Yêu cầu:

- Không tạo trước list toàn bộ các cửa sổ.
- `window_size > 0`.
- Nếu số phần tử không đủ cho một cửa sổ, không sinh giá trị nào.
- Có thể dùng `collections.deque`.

## Nhóm E: Bài tập tổng hợp

### Bài 16. Pipeline dữ liệu PM2.5 theo luồng

Cho dữ liệu thô:

```
raw_data = [
 "A01,12.5",
 "A02,-999",
 "A03,35.1",
 "A04,NA",
 "A05,41.8",
 "A06,20.6",
 "A07,30.2",
 "A08,"
]
```

Xây dựng pipeline generator:

```
read_lines(data)
parse_line(lines)
validate_records(records)
filter_high_pm25(records, threshold=25)
```

Yêu cầu:

- Không tạo list trung gian trong pipeline.
- `parse_line` trả về `(station_id, pm25)`.
- `validate_records` loại bản ghi không hợp lệ.
- `filter_high_pm25` giữ PM2.5 lớn hơn ngưỡng.
- In các trạm vượt ngưỡng.
- Tính số lượng, tổng, trung bình và lớn nhất của nhóm vượt ngưỡng.
- Viết docstring cho từng generator.

### Bài 17. Phân tích nhiều ngày quan trắc

Cho dữ liệu:

```
daily_pm25 = {
 "2026-09-10": [12.5, 18.2, 35.1, 28.4],
 "2026-09-11": [20.6, 41.8, 30.2, 15.0],
 "2026-09-12": [22.1, 24.5, 19.8, 38.6]
}
```

Yêu cầu:

1. Tạo generator lần lượt sinh: (date, value) cho từng giá trị PM2.5.
2. Lọc tất cả giá trị lớn hơn 25.
3. Tạo generator sinh: (date, daily\_mean)
4. Tìm ngày có PM2.5 trung bình cao nhất.
5. Dùng enumerate() in các lần vượt ngưỡng.
6. Dùng zip() ghép danh sách ngày và giá trị trung bình.
7. Không sử dụng Pandas hoặc NumPy.